

大作业

问题一：最短路径

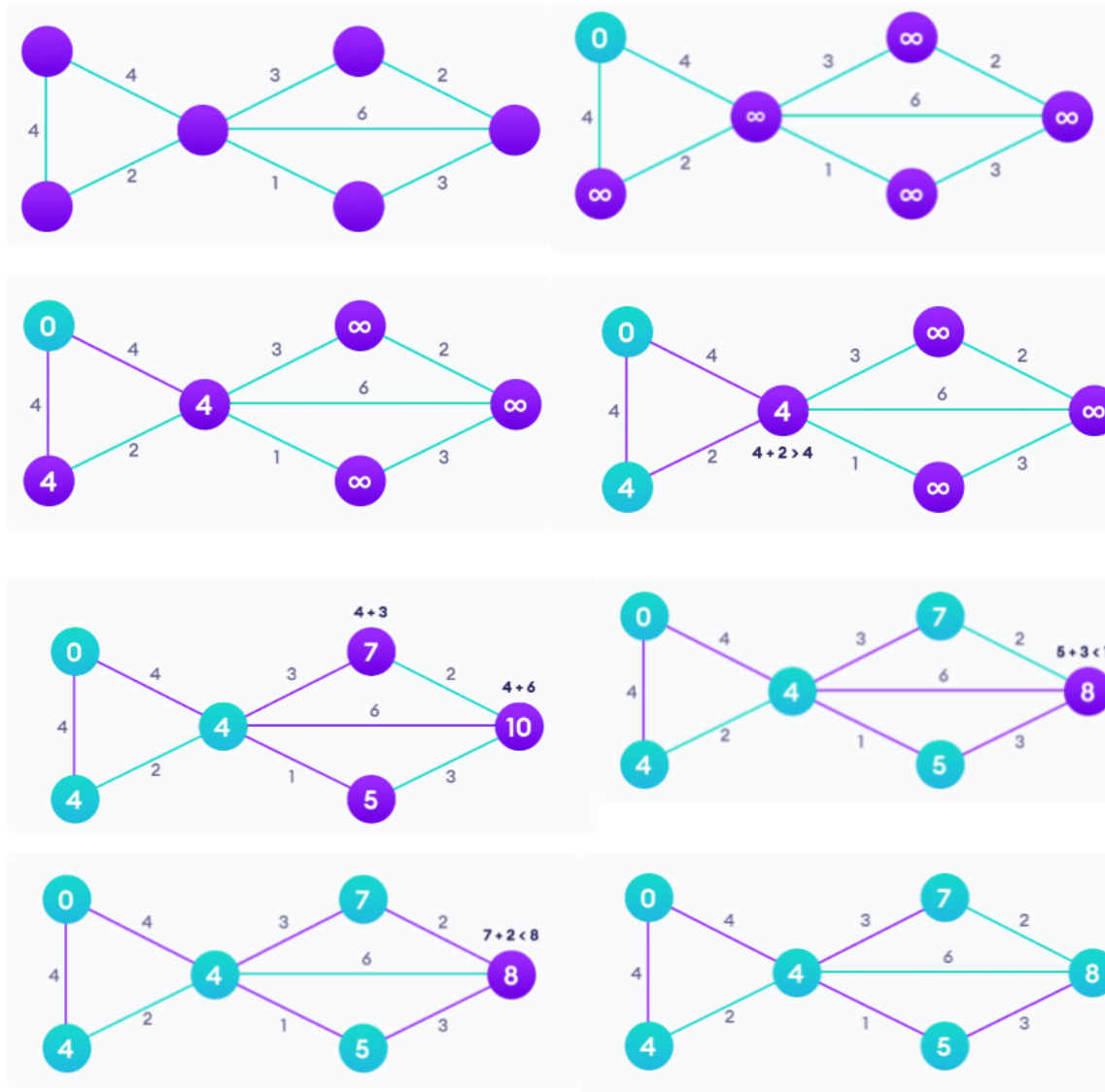
问题描述：

给定一个带权重的有向图和两个节点，找到连接这两个节点的最短路径。

求解算法描述

算法1：Dijkstra算法

- 从起始节点开始，逐步找到到达所有其他节点的最短路径。Dijkstra(迪杰斯特拉)算法是典型的单源最短路径算法，用于计算一个节点到其他所有节点的最短路径。主要特点是以起始点为中心向外层层扩展，直到扩展到终点为止。Dijkstra算法是很有代表性的最短路径算法，在很多专业课程中都作为基本内容有详细的介绍，如数据结构，图论，运筹学等等。注意该算法要求图中不存在**负权边**。
- 1)算法思想：设 $G=(V,E)$ 是一个带权有向图，把图中顶点集合 V 分成两组，第一组为已求出最短路径的顶点集合（用 S 表示，初始时 S 中只有一个源点，以后每求得一条最短路径，就将加入到集合 S 中，直到全部顶点都加入到 S 中，算法就结束了），第二组为其余未确定最短路径的顶点集合（用 U 表示），按最短路径长度的递增次序依次把第二组的顶点加入 S 中。在加入的过程中，总保持从源点 v 到 S 中各顶点的最短路径长度不大于从源点 v 到 U 中任何顶点的最短路径长度。此外，每个顶点对应一个距离， S 中的顶点的距离就是从 v 到此顶点的最短路径长度， U 中的顶点的距离，是从 v 到此顶点只包括 S 中的顶点为中间顶点的当前最短路径长度。
- 2)算法步骤：
 - a.初始时， S 只包含源点，即 $S = \{v\}$ ， v 的距离为0。 U 包含除 v 外的其他顶点，即： $U=\{\text{其余顶点}\}$ ，若 v 与 U 中顶点 u 有边，则 $\langle u,v \rangle$ 正常有权值，若 u 不是 v 的出边邻接点，则 $\langle u,v \rangle$ 权值为 ∞ 。
 - b.从 U 中选取一个距离 v 最小的顶点 k ，把 k ，加入 S 中（该选定的距离就是 v 到 k 的最短路径长度）。
 - c.以 k 为新考虑的中间点，修改 U 中各顶点的距离；若从源点 v 到顶点 u 的距离（经过顶点 k ）比原来距离（不经过顶点 k ）短，则修改顶点 u 的距离值，修改后的距离值的顶点 k 的距离加上边上的权。
 - d.重复步骤b和c直到所有顶点都包含在 S 中。



时间复杂度： $O((V + E) * \log V)$ 其中 V 是节点数， E 是边数， 优先队列再堆上经行操作

空间复杂度： $O(V + E)$

代码：

```
import time
import heapq

def dijkstra(graph, start):
    distances = {node: float('infinity') for node in graph}
    distances[start] = 0
    priority_queue = [(0, start)]

    # 采用优先队列
    while priority_queue:
        current_distance, current_node = heapq.heappop(priority_queue)

        if current_distance > distances[current_node]:
            continue

        for neighbor, weight in graph[current_node].items():
            new_distance = current_distance + weight
            if new_distance < distances[neighbor]:
                distances[neighbor] = new_distance
                priority_queue.append((new_distance, neighbor))
```

```

        distance = current_distance + weight

        if distance < distances[neighbor]:
            distances[neighbor] = distance
            heapq.heappush(priority_queue, (distance, neighbor))

    return distances

# 模拟输入
graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'C': 2, 'D': 5},
    'C': {'D': 1},
    'D': {}
}
start_node = 'A'

# 测试算法
start_time = time.time()
dijkstra_result = dijkstra(graph, start_node)
end_time = time.time()

print(f"Dijkstra Algorithm Time: {end_time - start_time:.6f} seconds")
print("Shortest Distances:", dijkstra_result)

```

算法2: Bellman-Ford算法

- 允许存在负权重的情况，通过对所有边进行松弛操作找到最短路径。核心思想：能否通过AB这条边使得起点到B的距离变得更短

最多松弛 $n-1$ 次，应为 n 个顶点的图，最多这个点有 $n-1$ 条边，至多 $n-1$ 次就可以找到最短路了

- 时间复杂度： $O(NM)$ N 顶点数， M 边数
- 空间复杂度： $O(N+M)$

优化: SPFA

SPFA (Shortest Path Faster Algorithm) 是对Bellman-Ford算法的一种优化，它使用了**队列**来减少**不必要的**松弛操作，提高算法的效率。

起点入队，从队列队首开始，扩展所有的队首相邻点，如果发现能够更新最短路就把另一个顶点入队，队首的所有边都遍历完了之后就出队，重复这个过程知道队列为空。

时间复杂度： $O(KE)$: K 是常数，平均值为2

```

import time
from collections import deque

def spfa(graph, start):
    distances = {node: float('infinity') for node in graph}
    distances[start] = 0
    queue = deque([start]) #初始点入队

```

```

in_queue = {node: False for node in graph}
in_queue[start] = True

while queue: #直到队列为空
    current_node = queue.popleft()
    in_queue[current_node] = False

    for neighbor, weight in graph[current_node].items():
        if distances[current_node] + weight < distances[neighbor]:
            distances[neighbor] = distances[current_node] + weight
            if not in_queue[neighbor]:
                queue.append(neighbor)
                in_queue[neighbor] = True

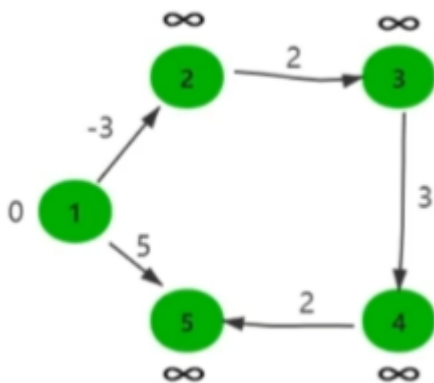
    return distances

# 模拟输入
graph = {
    'A': {'B': -3, 'E': 5},
    'B': {'C': 2},
    'C': {'D': 3},
    'D': {'E': 2},
    'E': {}
}
start_node = 'A'

# 测试算法
start_time = time.time()
spfa_result = spfa(graph, start_node)
end_time = time.time()

print(f"SPFA Algorithm Time: {end_time - start_time:.6f} seconds")
print("Shortest Distances:", spfa_result)

```



输入:

```

D:\enviroment\anaconda\envs\DL\python.exe D:\code\python\pycode\py_al\graph\bell-for.py
Bellman-Ford Algorithm Time: 0.000000 seconds
Shortest Distances: {'A': 0, 'B': -3, 'C': -1, 'D': 2, 'E': 4}

```

算法3: Floyd-Warshall算法

- 适用于带权重的有向图，通过动态规划找到所有节点之间的最短路径。
- Floyd-Warshall算法是一种用于解决所有节点对最短路径问题的经典算法。该算法采用动态规划的思想，通过逐步逼近最短路径的解，从而找到所有节点之间的最短路径。

1. 距离矩阵初始化

首先，算法需要构建一个距离矩阵，其中记录了图中每两个节点之间的直接距离。如果节点之间有边相连，则距离矩阵中对应的元素为边的权重，否则为无穷大。初始时，距离矩阵的对角线元素为0，表示节点到自身的距离为0。

2. 三重循环更新距离矩阵

Floyd-Warshall算法的核心是三重循环，通过中间节点的逐步迭代，更新距离矩阵中的最短路径。

具体步骤如下：

- 对于每一对节点 i 和 j ，以及每一个可能的中间节点 k ，检查是否存在一条路径 $i \rightarrow k \rightarrow j$ 的距离小于当前已知的 $i \rightarrow j$ 的距离。
- 如果存在这样的路径，更新距离矩阵中 i 到 j 的最短距离，即 $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$ 。

这个过程将在每次迭代中考虑一个新的中间节点，逐步更新距离矩阵，最终得到所有节点之间的最短路径。

- **时间复杂度：** 三重循环的迭代次数是 $O(V^3)$ ，其中 V 是节点的数量。因此，Floyd-Warshall算法的时间复杂度是 $O(V^3)$ 。
- **空间复杂度：** 需要维护一个距离矩阵，其大小为 $V \times V$ ，因此空间复杂度是 $O(V^2)$ 。

```
import time

def floyd_warshall(graph):
    nodes = list(graph.keys())
    n = len(nodes)
    distances = {node1: {node2: float('infinity') for node2 in nodes} for node1 in nodes}

    for node1 in graph:
        for node2, weight in graph[node1].items():
            distances[node1][node2] = weight

    for k in nodes:
        for i in nodes:
            for j in nodes:
                distances[i][j] = min(distances[i][j], distances[i][k] + distances[k][j])

    return distances

# 模拟输入
graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'C': 2, 'D': 5},
```

```
'C': {'D': 1},
'D': {}
}

# 测试算法
start_time = time.time()
floyd_warshall_result = floyd_warshall(graph)
end_time = time.time()

print(f"Floyd-Warshall Algorithm Time: {end_time - start_time:.6f} seconds")
print("Shortest Distances:", floyd_warshall_result)
```

输出:

```
Floyd-Warshall Algorithm Time: 0.000000 seconds
Shortest Distances:
A : {'A': 0, 'B': 1, 'C': 3, 'D': 4}
B : {'A': inf, 'B': 0, 'C': 2, 'D': 3}
C : {'A': inf, 'B': inf, 'C': 0, 'D': 1}
D : {'A': inf, 'B': inf, 'C': inf, 'D': 0}
```

问题二：排序问题

问题描述：

给定一个包含 n 个元素的数组（或列表、序列），排序算法的任务是以升序或降序的方式重新排列这些元素。

求解算法描述

算法1：冒泡排序

- 重复遍历数组，比较相邻的元素，如果顺序不对则交换，直到整个数组有序
- 时间复杂度： $O(N^2)$
- 空间复杂度： $O(1)$

代码：

```
def bubbleSort(nums):
    n = len(nums)
    for i in range(n):
        for j in range(0, n-i-1):
            if nums[j] > nums[j+1]:
                # 冒泡
                nums[j], nums[j+1] = nums[j+1], nums[j]
```

算法2：插入排序

- 将数组分为已排序和未排序两部分，每次从未排序部分取出一个元素，插入到已排序部分的正确位置.
- 算法复杂度：时间复杂度： $O(n^2)$
- 空间复杂度： $O(1)$ 。

代码

```
# 插入排序
def insertionSort(nums):
    for i in range(1, len(nums)):
        key = nums[i]
        j = i - 1
        # 插入元素 比较
        while j >= 0 and key < nums[j]:
            # 元素后移
            nums[j + 1] = nums[j]
            j -= 1
        # 进行插入
        nums[j + 1] = key
```

算法3：选择排序

- 每次选择数组中最小（或最大）的元素，放到已排序的部分的末尾。
- 时间复杂度： $O(n^2)$
- 空间复杂度： $O(1)$ 。

代码

```
# 选择排序
def selectionSort(nums):
    for i in range(len(nums)):
        min_index = i
        for j in range(i + 1, len(nums)):
            if nums[j] < nums[min_index]:
                # 选择一个
                min_index = j
        # 进行交换
        nums[i], nums[min_index] = nums[min_index], nums[i]
```

算法4：快速排序

- 选择一个基准元素，将数组划分为两部分，小于基准的放在左边，大于基准的放在右边，递归地对左右两部分进行排序。
- 时间复杂度： $O(n \log n)$ 平均情况， $O(n^2)$ 最坏情况,
- 空间复杂度： $O(\log n)$

代码

```
def quickSort(nums):
    if len(nums) <= 1:
        return nums
    # 枢纽元素
    pivot = nums[0]
    left = [x for x in nums[1:] if x <= pivot]
    right = [x for x in nums[1:] if x > pivot]
    return quickSort(left) + [pivot] + quickSort(right)
```

算法5: 归并排序

- 采用分治法，将数组分成两半，分别排序后再合并。
- 时间复杂度: $O(n \log n)$
- 空间复杂度: $O(n)$ (需要额外的空间来合并)

代码

```
def mergeSort(nums):
    if len(nums) > 1:
        mid = len(nums) // 2
        left_half = nums[:mid]
        right_half = nums[mid:]

        mergeSort(left_half)
        mergeSort(right_half)
        # i是left的指针 j是right的指针 nums进行合并
        i = j = k = 0

        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                nums[k] = left_half[i]
                i += 1
            else:
                nums[k] = right_half[j]
                j += 1
            k += 1
        # 多余部分直接复制
        while i < len(left_half):
            nums[k] = left_half[i]
            i += 1
            k += 1
        # 多余部分直接复制
        while j < len(right_half):
            nums[k] = right_half[j]
            j += 1
            k += 1
```


算法6: 堆排序

- 利用堆的性质进行排序，将数组构建成最大堆（或最小堆），然后反复将堆顶元素取出，直到整个数组有序。
- 时间复杂度： $O(n \log n)$
- 空间复杂度： $O(1)$

代码

```
def heapify(nums, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and nums[left] > nums[largest]:
        largest = left

    if right < n and nums[right] > nums[largest]:
        largest = right

    if largest != i:
        nums[i], nums[largest] = nums[largest], nums[i]
        heapify(nums, n, largest)

def heapSort(nums):
    n = len(nums)

    for i in range(n // 2 - 1, -1, -1):
        # 初始构造堆
        heapify(nums, n, i)

    for i in range(n - 1, 0, -1):
        # 将第一个和维护的堆最后一个交换
        nums[i], nums[0] = nums[0], nums[i]
        # 堆得数量减少
        heapify(nums, i, 0)
```

仿真

随机生成10000个数据，进行排序：

```
D:\enviroment\anaconda\envs\DL\python.exe D:\code\python\pycode\py_al\gaojisuanfa\sort.py
Bubble Sort Time: 3.916920 seconds
Quick Sort Time: 0.012434 seconds
Insertion Sort Time: 1.780994 seconds
Merge Sort Time: 0.018060 seconds
Selection Sort Time: 1.368284 seconds
Heap Sort Time: 0.025815 seconds

Process finished with exit code 0
```

其中冒泡排序最久，快速排序最快

